

Low-variance gradient estimation for mixture distributions

ReMix: reparameterization for mixtures

Команда проекта: Александр Уденеев, Алексей Пономарев,
Лина Горбунова, Максим Иванов

ИАД, МФТИ

Мотивация проекта

Для нормального распределения работает репараметризация:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Проблема

В смеси сначала выбирается дискретная компонента $c \sim \text{Cat}(\pi)$, затем $z \sim q_c$. Обычный backprop через этот выбор не даёт градиент по параметрам смеси

Цель

Реализовать и сравнить четыре несмещённых алгоритма для оценки градиента.

В PyTorch у `MixtureSameFamily` не поддерживается `rsample` для градиентов по параметрам.

Постановка задачи

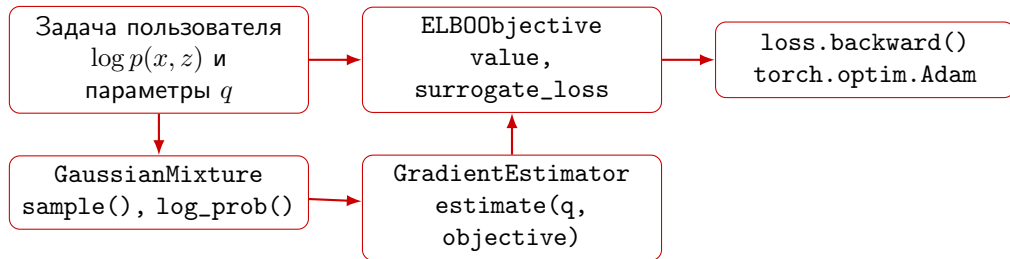
$$q_{\theta}(z) = \sum_{k=1}^K \pi_k q_{k,\theta}(z), \quad \pi = \text{softmax}(a), \quad q_{k,\theta} = \mathcal{N}(\mu_k, \text{diag } \sigma_k^2).$$

Максимизируем нижнюю оценку логарифма правдоподобия:

$$\mathcal{L}(\theta) = \mathbb{E}_{z \sim q_{\theta}}[f_{\theta}(z)], \quad f_{\theta}(z) = \log p(x, z) - \log q_{\theta}(z).$$

Обучаем $\theta = (a, \mu, \rho)$, где $\sigma = \text{softplus}(\rho) + \varepsilon$.

Классы и интеграция



Реализации интерфейса (расскажу ниже): ScoreFunction, Stratified, Implicit, PostStratified.

Стек: Python, PyTorch / torch.distributions, Pyro, autograd, pytest, torch.utils.benchmark, Matplotlib.

1. Score-function и модификация STL

Идея score-function: оценить, как параметры меняют плотность в случайной точке $z \sim q_\theta$.

$$\hat{g}_{\text{SF}} = (f_\theta(z) - b) \nabla_\theta \log q_\theta(z) + \partial_\theta f_\theta(z).$$

b не зависит от текущего z и уменьшает шум.

STL для репараметризованных алгоритмов оценки

В f заменяем $\log q_\theta(z)$ на $\log q_{\text{stopgrad}(\theta)}(z)$. Сохраняем градиент через z , убираем прямую производную по θ .

Источник: Roeder et al., 2017, §2–3

2. Stratified: сэмпл из каждой компоненты

- 1) Для каждого k берём $\varepsilon_k \sim \mathcal{N}(0, I)$.
- 2) Строим $z_k = T_{k,\theta}(\varepsilon_k) = \mu_k + \sigma_k \odot \varepsilon_k$.
- 3) Дифференцируем взвешенную сумму:

$$\hat{g}_{\text{strat}} = \nabla_{\theta} \sum_{k=1}^K \pi_k f_{\theta}(z_k).$$

Каждая компонента участвует в каждом шаге. Случайный выбор её номера заменяется суммированием. Получается несмещённая оценка.

Источник: Burroni, Sheldon, 2026, §3

3. Implicit: производная через фиксированный квантиль

- 1) Сэмплируем $z \sim q_\theta$ обычным способом.
- 2) Фиксируем его квантиль $u = F_\theta(z)$.
- 3) Неявно дифференцируем равенство $F_\theta(z) = u$:

$$F_\theta(z) = \sum_k \pi_k F_{k,\theta}(z), \quad \frac{\partial z}{\partial \theta} = -\frac{\partial_\theta F_\theta(z)}{q_\theta(z)}.$$

При изменении смеси точка движется так, чтобы её квантиль сохранялся. Получаем градиент и по компонентам, и по весам.

Источник: Figurnov et al., 2018, §3–4

4. Post-stratified: один сэмпл для всех компонент

Взять $z \sim q_\theta$ и вычислить вероятности его происхождения:

$$r_k(z) = \frac{\pi_k q_{k,\theta}(z)}{q_\theta(z)}, \quad v_k(z) = \left. \frac{\partial T_{k,\theta}(\varepsilon)}{\partial \theta} \right|_{\varepsilon=T_{k,\theta}^{-1}(z) \text{ фиксирован}}.$$

$$\hat{g}_{\text{PS}} = \partial_\theta f_\theta(z) + \sum_k r_k(z) \left[v_k(z)^\top \nabla_z f_\theta(z) + f_\theta(z) \nabla_\theta \log \pi_k \right].$$

Источник: Burroni, Sheldon, 2026, §4.3

Ожидаемый пример использования

```
import torch
from remix import DiagonalGaussianMixture, ELBOObjective
from remix.estimators import PostStratified

q = DiagonalGaussianMixture(n_components=4, event_dim=2)
objective = ELBOObjective(log_joint=target_log_prob, stl=True)
estimator = PostStratified(n_samples=32)
optimizer = torch.optim.Adam(q.parameters(), lr=1e-2)

for step in range(1000):
    optimizer.zero_grad()
    result = estimator.estimate(q.distribution(), objective)
    result.surrogate_loss.backward()
    optimizer.step()
```